Claude Code 原始碼深度分析報告

來自：AI超元域

我的B站頻道：https://space.bilibili.com/3493277319825652

專案： Claude Code CLI (Anthropic)

分析日期： 2026-03-31

原始碼快照： 2026年3月31日（npm source map 洩漏）

規模： 約1,900個檔案，512,000+行 TypeScript 程式碼

執行時： Bun | **UI框架：** React + Ink（深度定製 fork）| **CLI：** Commander.js

---

目錄

1. 概述總結
2. 整體架構
3. 核心引擎
4. 工具系統
5. 權限系統
6. 服務層
7. UI 架構
8. Bridge 橋接系統
9. 支撐基礎設施
10. 安全性分析
11. 設計模式與工程決策
12. 結論

---

1. 概述總結

Claude Code 是 Anthropic 官方的 CLI 工具，用於在終端中與 Claude 互動。它是一個深度工程化的代理式開發者工具，能夠編輯檔案、執行命令、搜尋程式碼庫、以及協調多代理工作流。

核心技術亮點：

- **自定義 Ink fork** — 包含 React reconciler、Yoga 佈局引擎、雙緩衝渲染、滑鼠支援
- **5層錯誤恢復** — 核心查詢迴圈中最大化會話存活率
- **43個工具** — 統一型別系統、逐工具權限模型、串流並行執行
- **多提供商 API** — 支援 Anthropic 直連、AWS Bedrock、Azure Foundry、Google Vertex、Claude.ai
- **8種 MCP 傳輸協議** — 用於外部工具整合
- **編譯時死程式碼消除** — 透過 Bun 的 `feature()` 標誌清除15+個內部功能
- **純 TypeScript 移植** — Yoga (C++)、nucleo/fzf (Rust)、syntect/bat (Rust)

---

2. 整體架構

2.1 頂層資料流

```
main.tsx (Commander.js CLI) --> REPL
  --> QueryEngine.submitMessage() --> query()
  --> callModel() --> StreamingToolExecutor
  --> Stop Hooks
```

2.2 目錄結構

目錄	檔案數	用途
`tools/`	43個子目錄	代理工具實作
`commands/`	~50個子目錄	斜線命令實作
`components/`	~140個檔案	Ink UI 元件
`hooks/`	~85個檔案	React Hooks
`services/`	~38個條目	外部服務整合

目錄	檔案數	用途
`utils/`	100+個檔案	工具函式
`bridge/`	~30個檔案	IDE 與遠端控制橋接
`constants/`	~20個檔案	設定常數
`ink/`	~90個檔案	自定義 Ink 渲染引擎
`state/`	~8個檔案	狀態管理
`skills/`	~5個檔案	技能系統
`memdir/`	~8個檔案	持久化記憶
`keybindings/`	~14個檔案	鍵盤綁定處理
`vim/`	~5個檔案	Vim 模式模擬

2.3 技術棧

類別	技術
語言	TypeScript（嚴格模式）
執行時	Bun
終端 UI	React + Ink（深度定製 fork）
CLI 解析	Commander.js (extra-typings)
Schema 驗證	Zod v4
程式碼搜尋	ripgrep
協議	MCP SDK、LSP
API	Anthropic SDK
遙測	OpenTelemetry + gRPC
特性開關	GrowthBook
認證	OAuth 2.0、JWT、macOS Keychain

3. 核心引擎

3.1 QueryEngine.ts（約46,000行）— LLM 會話引擎

`QueryEngine` 類管理整個會話的生命週期。每個會話一個實例，`submitMessage()` 是核心入口。

關鍵設計決策：

- **AsyncGenerator 模式**：`submitMessage()` 是 `async*` 生成器，串流輸出 `SDKMessage`
- **提前持久化 transcript**：使用者訊息在 API 呼叫前即寫入，即使程序被終止也能恢復
- **權限拒絕追蹤**：包裝 `canUseTool` 回呼以攔截並上報拒絕
- **即發即忘 transcript**：助手訊息使用 `void recordTranscript()` 寫入，避免阻塞

submitMessage 流程：

1. 提取配置，設定 CWD，確定模型和 thinking 配置
2. 取得系統提示詞元件（含 coordinator 使用者上下文）
3. 建構 `ProcessUserInputContext`，處理使用者輸入
4. 將使用者訊息持久化到 transcript（提前寫入）
5. 處理孤立權限（每個引擎生命週期僅一次）
6. 產出 `buildSystemInitMessage`（工具、MCP 客戶端、模型資訊）
7. 進入 `query()` 迴圈，分發每種訊息類型
8. 透過 `accumulateUsage` 追蹤用量
9. 記錄 transcript（助手訊息即發即忘）
10. 檢查預算限制、結構化輸出重試次數、最大輪次
11. 產出最終結果訊息（費用、用量、耗時）

3.2 query.ts（約1,730行）— Claude Code 的心臟

核心代理迴圈採用 `while(true)` 模式 + 顯式 State 狀態轉換（非遞迴，避免長會話堆疊溢位）。

每輪迭代流程（14步）：

1. 技能發現預取 — 在模型串流輸出期間並行執行
2. 查詢鏈追蹤 — 建立/遞增 `chainId` / `depth`
3. 工具結果預算 — `applyToolResultBudget` 強制每條訊息聚合大小
4. Snip 壓縮 — 修剪過期歷史（特性門控）

5. 微壓縮 — 應用細粒度上下文編輯
6. 上下文摺疊 — 投影摺疊後的上下文檢視（特性門控）
7. 自動壓縮 — token 計數過高時主動壓縮
8. 模型串流呼叫 — 呼叫 `deps.callModel()`，處理串流事件
9. 工具執行 — `StreamingToolExecutor` 在串流輸出期間即開始執行
10. 錯誤處理 — 多層恢復級聯
11. Stop Hooks — 後取樣鉤子、記憶提取
12. 工具結果 — 收集並作為附件注入
13. 工具池重整 — 為新連線的 MCP 伺服器重整工具池
14. 最大輪次檢查 — 超出則產出 `max_turns_reached`

5層錯誤恢復機制：

層級	策略	觸發條件
第1層	上下文摺疊排水	Prompt 過長（最低成本）
第2層	響應式壓縮	最大輸出升級 8K → 64K tokens
第3層	多輪恢復	推動訊息（最多3次）
第4層	模型回退	切換到備用模型
第5層	完整會話摘要	—

3.3 特性開關架構

機制	時機	方式
`feature()` (bun:bundle)	編譯時	打包器死程式碼消除
Statsig/環境變數門控	執行時	每次查詢快照一次

---

4. 工具系統

4.1 工具型別系統（Tool.ts，約29,000行）

每個工具遵循 `Tool<Input, Output, Progress>` 介面。`buildTool()` 工廠函式應用安全預設值（fail-closed：`isConcurrencySafe=false`，`isReadOnly=false`）。

4.2 核心工具清單

工具	檔案數	複雜度
BashTool	18	最高 — Shell 安全、沙盒、ML 分類器
AgentTool	17	第二高 — 多模式代理生成
FileEditTool	6	最嚴格的輸入驗證
FileReadTool	5	支援5種檔案類型，token 限制
FileWriteTool	3	必須先讀後寫
GlobTool	3	基於 ripgrep 的模式比對
GrepTool	3	3種輸出模式，分頁支援
SkillTool	4	技能到代理的橋接
MCPTool	4	執行時模板覆寫

4.3 工具執行生命週期

1. Zod Schema 驗證（`safeParse`）
2. 自定義驗證（`tool.validateInput`）
3. 工具存在性檢查（限制代理可見工具）
4. `PreToolUse` 鉤子（可變更輸入）
5. 權限決策（hooks → `tool.checkPermissions` → 使用者確認）
6. 工具呼叫（`tool.call`）
7. `PostToolUse` 鉤子
8. 結果清理（截斷、工具結果預算）

4.4 BashTool 安全層級

層級	元件	功能
第1層	`bashSecurity.ts`	命令注入、Zsh 漏洞、IFS 注入、Unicode 混淆
第2層	`readOnlyValidation.ts`	git/find/grep/ls 唯讀命令白名單
第3層	`bashPermissions.ts`	模式驗證、路徑限制、複合命令拆分
第4層	`shouldUseSandbox.ts`	macOS 沙盒判定

層級	元件	功能
第5層	ML 分類器	基於推理的命令風險分類

4.5 FileEditTool 驗證鏈（12步）

1. 團隊記憶檔案中的機密偵測
2. `old_string != new_string` 檢查
3. 拒絕規則檢查 + UNC 路徑保護
4. 檔案大小限制（1 GiB）
5. 編碼偵測（UTF-8 / UTF-16LE）
6. 檔案存在性檢查（含相似檔案建議）
7. Jupyter notebook 重定向
8. 必須先讀後寫強制執行
9. 過期偵測（含內容比較回退）
10. 引號正規化（處理彎引號）
11. 多重比對偵測（除非 `replace_all` 否則拒絕）
12. 設定檔案編輯驗證

---

5. 權限系統

5.1 三層處理器級聯

```
hasPermissionsToUseTool
  |-- allow --> 直接執行
  |-- deny  --> 拒絕
  |-- ask   --> 使用者確認介面：
  Coordinator（協調器）
    --> 自動核准（已儲存規則）
    --> Bash 命令分析（複合命令拆分）
  Swarm Worker（群組工作者）
    --> 上報給 leader（附帶上下文）
    --> 等待（非阻塞佇列）
  React ToolUseConfirm
    --> 200ms 防抖
    --> Bridge/遠端委託
```

5.2 權限上下文

`ToolPermissionContext`（`DeepImmutable` 不可變）：

- **mode**：`default` | `auto` | `plan` | `acceptEdits` | `bypassPermissions` | `dontAsk` | `bubble`
- `alwaysAllowRules` / `alwaysDenyRules` / `alwaysAskRules`
- `additionalWorkingDirectories`（額外工作目錄）
- `shouldAvoidPermissionPrompts`（用於背景代理）

5.3 受保護路徑

`.gitconfig`、`.bashrc`、`.zshrc`、`.profile`、`.mcp.json`、`.claude.json`、`.git/`、`.vscode`

---

6. 服務層

6.1 API 服務 — 多提供商工廠

提供商	SDK	說明
Anthropic 直連	`@anthropic-ai/sdk`	預設，OAuth 或 API key
AWS Bedrock	`@anthropic-ai/bedrock-sdk`	動態匯入
Azure Foundry	`@anthropic-ai/foundry-sdk`	動態匯入
Google Vertex AI	`@anthropic-ai/vertex-sdk`	動態匯入
Claude.ai	—	OAuth bearer，訂閱使用者存取

重試策略：

- 指數退避 + 抖動
- 529 僅對前景查詢重試，防止級聯放大
- 無人值守持久重試模式（429/529 無限重試）
- AWS/GCP 憑證重整整合到重試迴圈中

6.2 MCP 服務

8種傳輸類型： `stdio`、`sse`、`sse-ide`、`ws`、`ws-ide`、`http`、`sdk`、`claudeai-proxy`

設定作用域： `local` → `user` → `project` → `dynamic` → `enterprise` → `claudeai` → `managed`

6.3 上下文壓縮 — 三層

層級	觸發條件	機制
微壓縮	每輪	對單條訊息進行細粒度快取編輯
自動壓縮	tokens > contextWindow - 13K	Fork 代理共享 prompt cache
完整壓縮	手動 `/compact`	完整會話摘要

6.4 分析與認證

- **零依賴入口**：事件排隊直到 `attachAnalyticsSink()` 被呼叫
- **PII 路由**：`_PROTO_*` 前綴 → 特權 BigQuery 欄
- **GrowthBook**：磁碟快取過期讀取 + 遠端重整
- **OAuth**：PKCE (S256) 授權碼流程，支援 Claude.ai 和 Console 登入

---

7. UI 架構

7.1 渲染堆疊 — 自定義 Ink Fork（90+個檔案）

元件	技術
React Reconciler	react-reconciler（Concurrent 模式）
佈局引擎	Yoga（純 TypeScript 移植）
渲染	雙緩衝幀、基於差異的終端寫入
輸入	自定義 parse-keypress、滑鼠 hit-testing
文字選取	Alt-screen 模式 + 搜尋覆蓋層

7.2 元件層級

```
cli.tsx
  main.tsx（並行預取：MDM、Keychain、API 連線）
  （Onboarding → Trust → MCP 設定）
      App.tsx > FpsMetricsProvider > StatsProvider
        > AppStateProvider > MailboxProvider > VoiceProvider
          > REPL.tsx（~5,000行 — 主要互動迴圈）
```

7.3 狀態管理 — 極簡自研（僅35行）

不使用 Redux 或 Zustand。`createStore<T>()` 返回 `getState` / `setState` / `subscribe`。`AppState` 約570行類型定義，透過 `useAppState(selector)` + `useSyncExternalStore` 實現最小化重渲染。

7.4 輸入處理管線

```
stdin → Ink parse-keypress
  → 按鍵正規化（死鍵合成、chord、1000ms 逾時）
    → Vim 模式（normal/insert、d/c/y、h/l/w/b/e/$）
      → 編輯器（kill ring、yank、history、multiline）
```

---

8. Bridge 橋接系統

8.1 REPL Bridge（本地 CLI → Claude.ai）

版本	讀取傳輸	寫入傳輸
v1	WebSocket	—
v2	SSE	HTTP POST 到 Session-Ingress

8.2 Remote Bridge

CCR v2 `/worker/*` 端點

模式	隔離方式
`single-session`	在 cwd 中的單個會話
`worktree`	每個會話使用獨立的 git worktree
`same-dir`	所有會話共享同一目錄

8.3 訊息協議與 IDE 整合

- **入站**：`control_response`（權限決策）、`control_request`（伺服器命令）、user 訊息
- **出站**：僅使用者/助手訊息，虛擬訊息被過濾

IDE 透過 MCP `sse-ide` / `ws-ide` 傳輸連線，支援程式碼選取轉發、檔案 `@` 提及、差異顯示、權限回呼。

---

9. 支撐基礎設施

9.1 持久化記憶

類型	用途	範例
`user`	使用者畫像、偏好	深度 Go 經驗，React 新手
`feedback`	行為指導	整合測試必須使用真實資料庫
`project`	進行中的工作上下文	合併凍結從 2026-03-05 開始
`reference`	外部系統指標	Pipeline bugs 在 Linear 專案 INGEST 中

9.2 純 TypeScript 原生模組移植

模組	替代目標	用途
`yoga-layout`	Meta Yoga (C++)	Ink 的 Flexbox 佈局
`file-index`	nucleo (Rust)	fzf-v2 模糊搜尋，270K+ 檔案非同步索引
`color-diff`	syntect+bat (Rust)	highlight.js 語法高亮 + 詞級差異

9.3 其他支撐系統

- **CCR 上游代理**：TCP-to-WebSocket CONNECT 中繼，`prctl` 防 ptrace，Protobuf 分幀，全部 fail-open
- **伴侶精靈**：18種物種、6種眼睛、8種帽子、5種稀有度，Mulberry32 PRNG 確定性生成

- **設定子系統**：`policySettings` > `projectSettings` > `localSettings` > `userSettings`，安全欄位受限
- **Swarm 團隊系統**：InProcess/Tmux/iTerm 後端，Leader-Worker 權限委派
- **設定遷移**：11個冪等遷移（模型別名、權限模式、設定路徑）
- **優雅關閉**：SIGINT/SIGTERM/SIGHUP 處理，30秒孤立偵測，安全強制退出

---

10. 安全性分析

10.1 BashTool 安全（縱深防禦）

層級	緩解的威脅
Shell 安全分析	命令注入（`$()`、反引號）、程序替換
Zsh 專項防護	`zmodload`、`syswrite`、`ztcp` 利用
輸入清理	IFS 注入、花括號展開、Unicode 空白字元混淆
NTLM 保護	UNC 路徑在所有檔案系統工具上阻止
唯讀白名單	Git 讀取命令、docker inspect、gh 讀取命令
macOS 沙盒	按命令可設定的沙盒
ML 分類器	基於推理的命令風險分類
路徑驗證	拒絕專案外的寫入、危險檔案路徑

10.2 全棧安全措施

- **檔案系統**：必須先讀後寫、過期偵測、裝置路徑阻止、UNC 保護、機密偵測
- **憑證**：`prctl` 防 ptrace、token 僅堆記憶體、OAuth PKCE、JWT、macOS Keychain
- **Prompt 注入**：工具結果預算限制、結構化輸出鉤子、訊息去重防重放
- **PII**：類型安全標記防誤記、`_PROTO_*` 前綴路由到特權儲存

---

11. 設計模式與工程決策

11.1 架構模式

模式	應用位置	目的
AsyncGenerator	`submitMessage()`、`query()`	串流訊息消費
Fail-closed 預設	`buildTool()`	安全優先的工具註冊
依賴注入	`QueryDeps`、`ToolUseContext`	可測試性
編譯時 DCE	`feature()` + `bun:bundle`	剝離內部功能
品牌類型	`SessionId`、`AgentId`	防止 ID 混淆
零依賴入口	Analytics `index.ts`	打破匯入迴圈
雙緩衝渲染	Ink Output 系統	無閃爍終端更新
Prompt 快取穩定性	`assembleToolPool()` 排序	防快取失效

11.2 效能最佳化

最佳化	影響
啟動時並行預取	MDM、Keychain、API 預連線並行於模組評估
串流工具執行	模型輸出期間即執行工具，降低延遲
惰性匯入	重量級模組首次使用時載入
虛擬捲動	僅掛載可見範圍內的訊息
LRU 快取 JSON 解析	50條目快取
即發即忘 transcript	助手訊息非阻塞寫入
Blit 最佳化	僅寫入變化的終端單元格

11.3 可靠性模式

模式	應用位置	效果
斷路器	自動壓縮（最多3次連續失敗）	—
Fail-open 設計	代理、語音、記憶、分析	—

模式	應用位置	效果
指數退避 + 抖動	API 重試	級聯放大防護
孤立程序偵測	30秒間隔程序監控	—
優雅降級	529 僅對前景查詢重試	—
—	所有支撐系統不致命	—

---

12. 結論

12.1 工程品質評估

- **架構**：查詢引擎、工具系統、權限、UI 和服務之間有清晰的分離。`QueryDeps` 注入模式和 `ToolUseContext` 提供了可測試、可組合的抽象。
- **效能**：在每一層都有積極的並行化 — 啟動預取、串流工具執行、並行工具批次、虛擬捲動和基於差異的渲染。
- **安全**：Shell 執行的縱深防禦（6+層）、fail-closed 預設值、NTLM 保護、過期追蹤和 PII 安全分析。
- **可靠性**：核心迴圈5層錯誤恢復、斷路器、fail-open 支撐系統、帶孤立偵測的優雅關閉。
- **開發者體驗**：自定義 Ink fork 提供桌面級終端 UI，支援滑鼠、Vim 和鍵盤 chord。

12.2 突出技術成就

1. **純 TypeScript Yoga 移植** — 消除原生二進位制依賴，實現跨平台佈局
2. **串流工具執行** — 重疊模型推理和工具執行以降低延遲
3. **編譯時特性門控** — 透過 Bun 的 `feature()` 系統實現內部/外部建置分離
4. **多提供商 API 抽象** — 跨5個雲端提供商的統一介面
5. **35行狀態管理** — 證明複雜狀態可以不依賴框架來管理

12.3 規模指標

指標	數值
總檔案數	約1,900

指標	數值
總程式碼行數	512,000+
工具實作	43個
斜線命令	約50個
UI 元件	約140個
React Hooks	約85個
服務模組	約38個
MCP 傳輸類型	8種
特性開關	15+（編譯時）
伴侶精靈物種	18種

本報告基於2026年3月31日的原始碼快照生成。僅用於教育和安全研究目的。

來自：AI超元域 | B站頻道